

DRLLog: Deep Reinforcement Learning for Online Log Anomaly Detection

Junwei Zhou, Yuyang Gao, Ying Zhu, Xiangtian Yu, Yanchao Yang, Cheng Tan and Jianwen Xiang

Abstract—System logs record the system's status and application behavior, providing support for various system management and diagnostic tasks. However, existing methods for log anomaly detection face several challenges, including limitations in recognizing current types of anomalous logs and difficulties in performing online incremental updates to the anomaly detection models. To address these challenges, this paper introduces DRLLog, which applies Deep Reinforcement Learning (DRL) networks to detect anomalous events. DRLLog uses Deep Q Network (DQN) as the agent, with log entries serving as reward signals. By interacting with the environment generated from log data and adopting various action behaviors, it aims to maximize the reward value obtained as feedback. Through this approach, DRLLog achieves learning from historical log data and perception of the current environment, enabling continuous learning and adaptation to different log sequence patterns. Additionally, DRLLog introduces low-rank adaptation by using two low-rank parameter matrices in the fully connected layer of the DQN to represent changes in its weight matrix. During online model learning, only low-rank parameter matrices of the model are updated, effectively reducing the model's overhead. Furthermore, DRLLog introduces focal loss to focus more on learning the features of anomalous logs, effectively addressing the issue of imbalanced quantities between normal and anomalous logs. We evaluated the performance on widely used log datasets, including HDFS, BGL and ThunderBird, showing an average improvement of 3% in F1-Score compared to baseline methods. During online model learning, DRLLog achieves an average reduction of 90% in parameter count and a significant decrease in training and testing time as well.

Index Terms—Log anomaly detection, Deep reinforcement learning, Low rank adaptation, Focal loss.

I. INTRODUCTION

MODERN computer systems are large-scale and provide a wide range and variety of online services and intelligent applications, which are core components of the information industry. These systems need to maintain high availability and reliability to avoid financial losses and application crashes. In large-scale system event management, anomaly detection plays a key role, and its goal is to discover abnormal behavior of the system in time to reduce system downtime [1]. Almost all modern computer systems have deployed logging systems to record system runtime information in detail and support system management and diagnostic tasks [2]. Therefore, anomaly detection methods based on system

logs have become a research topic of concern in academia and industry.

Traditional anomaly detection methods are usually based on standard mining methods [3]. However, these traditional methods have many drawbacks, for example: (1) Domain knowledge is required. They depend heavily on expert-defined rules or regular expressions, which are time-consuming to create and often cannot keep up with evolving systems; (2) It is difficult to identify unknown faults or can only identify a single type of anomalous logs. The traditional log anomaly detection methods based on explicit keyword or regular expression matching can accurately detect the exception logs. However, with the continuous update of the system, some new fault types may be generated that are not sufficient to match the existing patterns; (3) The anomaly detection model cannot be updated incrementally in an online manner. They are not timely enough to adapt to new data or mode changes, which affects the timeliness and accuracy of detection, and is difficult to cope with dynamic environments and constantly changing log data, resulting in gradual failure of the model. Therefore, with the rise of machine learning and deep learning, researchers have started exploring their application in log anomaly detection.

In terms of traditional machine learning methods, Xu et al. [4] created a count vector of log events within a specific time window and used principal component analysis (PCA) to reduce its dimensionality while preserving key features. Lou et al. [5] introduced an automated method to extract system invariants from console logs, enabling high-precision anomaly detection and dynamic adaptation to system changes. Lin et al. [6] designed a cluster-based log anomaly detection method called LogCluster to identify log entries that do not conform to any existing log templates.

Deep learning based log anomaly detection methods automatically extract complex features from raw log data, leveraging their strong expressive power and superior generalization ability to become the mainstream approach. The DeepLog method proposed by Du et al. [7] treated logs as sequences that follow specific patterns and syntax rules. It utilized Long Short Term Memory (LSTM) networks to learn log sequence patterns from normal log execution paths, thereby capturing nonlinear high-dimensional dependencies between log sequences. Meng et al. [8] designed the Template2Vec, a method that converts log templates into high-dimensional vectors using word embedding to extract semantic and syntactic information. And Zhang et al. proposed LogRobust [9], which uses term frequency-inverse document frequency (TF-IDF) to weight-average word vectors, obtaining log vector representations that highlight key information.

Junwei Zhou, Yuyang Gao, Ying Zhu, Xiangtian Yu, Yanchao Yang, Cheng Tan and Jianwen Xiang were with the Sanya Science and Education Innovation Park, the School of Computer Science and Artificial Intelligence, Wuhan University of Technology, Wuhan, China, 430070. Corresponding author: Jianwen Xiang (E-mail: jwxiaang@whut.edu.cn).

Manuscript received February 13, 2025.

Although these methods have been successful in certain situations, they still have some shortcomings. For example, the training of model often relies on a large amount of historical logging data, and over time, the system may introduce new features or change configurations, which can lead to new logging patterns. Therefore, in order to maintain the effectiveness of the model, it is necessary to regularly update the training data set and retrain the model. At the same time, both DeepLog and LogRobust need to convert unstructured log data into some form of structured data (such as event templates). This transformation process may lose some important contextual information, affecting the effectiveness of the model. In addition, how to effectively and accurately detect abnormal events from a large number of complex logging data is still a great challenge. With the complexity of computer systems and applications, abnormal events are relatively rare in real log data, resulting in an imbalance between normal categories and abnormal categories, affecting model training and performance evaluation.

Deep Reinforcement Learning (DRL) exhibits significant advantages in the field of log anomaly detection due to its ability to handle complex, dynamic environments and perform end-to-end learning on unstructured data. Kinza Arshad et al. [10] presented a comprehensive review of DRL models in the context of anomaly detection applications. The review analyzed various DRL frameworks, proposed DRL methods, and performance comparisons with alternative approaches. In this review, 50 different datasets for anomaly detection experiments were identified, along with 17 different DRL models for detecting anomalies. Duan et al. [11] introduced a log anomaly detection method, QLLog, based on Q-learning. It aims to detect various types of system anomalies and rank their severity in an ordered manner. First, they established a mathematical model for log anomaly detection, demonstrating that log anomaly detection is essentially a sequential decision problem. Then, they utilized the Q-learning algorithm to construct the core of the anomaly detection model. Finally, combining the Q-learning algorithm with specifically designed rules, the model can detect anomalies when log patterns deviate from normal execution.

Traditional anomaly detection methods face challenges in adapting to changing system environments, the tediousness of manual rule and feature adjustments, and dependence on structured data. DRL addresses these shortcomings by possessing the ability to handle complex environments, learn complex system behavior patterns from log data, and adapt to diverse and dynamic log information. This adaptability is crucial for understanding the diversity and dynamics present in system logs, enhancing the model's comprehensiveness. Furthermore, DRL exhibits adaptability, enabling learning and adjustment in continuously changing system environments without manual intervention. This adaptability allows the model to cope with new log patterns and system changes, providing increased robustness. Additionally, DRL can learn and capture long-term dependencies in log data, especially in models based on Recurrent Neural Networks (RNN) or LSTM. This capability enables the model to better understand the temporal correlations between log entries, resulting in

more accurate detection of abnormal behavior. Moreover, some DRL models support real-time learning and allow anomaly detection in real-time system logs, which is very beneficial for quickly adapting to new log modes, handling real-time changing system environments, and improving the real-time performance of the system.

In log anomaly detection, log data is used as the environment of the agent, each log entry is regarded as a state, and the action of the agent corresponds to the anomalous label of each log entry. By designing a reward mechanism, during the training process, log entries are used as reward signals and different actions are taken to guide the agent to learn the correct anomaly detection strategy. Rewards are designed based on real anomaly detection situations to encourage agents to improve the accuracy of identifying anomalies. DRL exhibits adaptability and can learn optimal anomaly detection strategies from dynamic changes in log data. Through deep learning models, we can better capture the complex relationships between log entries and improve the accuracy of anomaly detection. At the same time, the anomaly detection strategy can be dynamically adjusted to adapt to log mode changes in different environments, thereby enhancing robustness.

The contributions of this paper are summarized as follows:

- 1) This paper introduces the application of the Deep Q Network (DQN) of DRL to log anomaly detection, proposing the online log detection model, DRLLog. By combining deep learning and Q-learning, the model effectively makes decisions and learns through a neural network, achieving efficient online log anomaly detection.
- 2) DRLLog introduces the concept of low-rank adaptation by adding a bypass in the structure of the neural network, representing parameter matrices in a low-rank form. This effectively reduces the model's complexity and, to some extent, reduces the demand for computational resources.
- 3) DRLLog introduces the concept of focus loss, combined with mean square error loss as the loss function, making the model more focused on learning the features of anomalous logs.
- 4) Extensive experiments on three public log datasets (HDFS, BGL and ThunderBird) show that when there is limited training log data, DRLLog, compared to major advanced anomaly detection models, achieves good performance with resource consumption below 90%. It also reduces training time by over 50% and testing time by 60%-75%.

The remainder of this paper is organized as follows. We present related work in Section II, in Section III we present the design and details of DRLLog, and in Section IV we describe and discuss the evaluation experiments. Finally, we conclude our work in Section V. The source code of DRLLog is available at: <https://github.com/YuxiangtianT/DRLLog>.

II. RELATED WORK

A. Data Preprocessing and Word Embedding

In log anomaly detection, the importance of log preprocessing lies in effectively cleaning, structuring, and optimizing raw

log data through a series of steps to improve the performance of the anomaly detection model [12]. Parsing and structuring operations parse raw log messages into structured data according to a specific format, making the data easier to understand and analyze. And through feature extraction, representative features are extracted from the parsed data, which helps to reduce the dimensionality of the data, improve the training efficiency of the model, and ensure that the model can focus on key information related to anomaly detection.

At the same time, word embedding technology [13] converts log messages into vector representations. Word embedding retains semantic information and provides richer and more abstract input for the model. Commonly used algorithms include Word2Vec [14], GloVe [15] and FastText [16], etc. By learning embedding representations, the model can gain a deeper understanding of the semantic information of log data, thereby improving the model's generalization ability and making it more adaptable and able to effectively cope with the detection challenges of unknown anomalies.

B. Deep Reinforcement Learning

Deep reinforcement learning has attracted much attention in the field of machine learning and has become a very popular technology due to its excellent performance [17]. DRL uses a method that combines deep neural networks and reinforcement learning to interact between the agent and the environment and achieve the goal by learning the optimal strategy [18].

Hsu et al. [19] proposed an anomaly network intrusion detection system based on DRL, which possesses self-updating capabilities to reflect new types of network traffic behavior. Pang et al. [20] introduced a method actively seeking new anomalous categories beyond labeled training data. This method learns to utilize existing data models and explore anomalies in new categories. Wu et al. [21] proposed a novel semi-supervised sequential anomaly detection algorithm that efficiently learns and adapts to real-world time series data anomalies using DRL and active learning. Zha et al. [22] presented an active anomaly detection framework with a meta-policy, utilizing DRL to train the meta-policy to select the most appropriate instances and explicitly optimize the discovery of anomalies throughout the entire query process.

The basic structure of DRL includes environment, agent, state, action, reward, strategy, value function and model [23]. The agent learns through interaction with the environment and continuously optimizes itself with the help of feedback and rewards provided by external systems. Among them, the state describes the information of the environment, the action is the operation that the agent can perform in a certain state, and the reward is the feedback signal provided by the environment, which is used to evaluate the behavioral effect of the agent. State, action and reward jointly describe the interaction process between the agent and the environment. The agent selects an action based on the current state and obtains the corresponding reward signal. A policy defines a probability distribution for selecting actions in a given state and serves as the agent's decision rule. Value functions are used to evaluate the value of a state or state-action pair and

guide decision optimization during the learning process. In addition, the model is responsible for estimating the dynamic properties of the environment to provide support for planning and forecasting. Overall, these elements jointly build the basic framework of the DRL system [24].

III. METHODOLOGY

In our work, we aim to build an online log anomaly detection model, and are committed to reducing the number of parameters during model training to reduce training overhead and effectively deal with the imbalance of normal and abnormal log data to improve model performance. Our framework is divided into two main parts: the data preprocessing and word embedding part, and the online log anomaly detection model part based on the DQN of deep reinforcement learning. The DRLLog architecture is shown in Fig. 1. In the first part, we extract log templates from log entries to form log sequences, and then use word embedding methods to generate log vector representations to preserve the semantic information of the logs. Then, we use the log vector to build an experience pool memory library for repeated learning to improve the generalization ability of the model. In the second part, the DRLLog model includes two networks with the same structure but different parameters: the target network and the prediction network. In order to reduce the model complexity, we introduce side channels in the network structure and modify the parameter matrix through low-rank adaptive representation. Subsequently, DRLLog randomly extracts log vectors from the memory bank, inputs them into the target network and prediction network, calculates the difference between the outputs of the two networks through mean square error loss and focus loss, and uses it to train the prediction network, thereby updating the model parameters. After a certain number of epochs of training, the prediction network parameters are copied to the target network. Finally, log anomaly detection is implemented by comparing the predicted value output by the model with the set threshold. In addition, we use different proportions of training logs to train the model, and simulate online log anomaly detection scenarios by incrementally updating the anomaly detection model.

A. Data Preprocessing and Word Embedding

The section first parses unstructured, text-free log data into a structured representation, extracts log templates from the log data, and then matches log entries to the template.

Consider the following log execution sequence $L = (S, E)$, where S is the set of states in the log (with n log states, $n \in \mathbb{N}$), defined as $S = \{s_1, s_2, \dots, s_n\}$, where $\forall s_i \in S$ represents a log state. The set $E = S \times S$ is the set of all log execution paths, where each path $e_{xy} \in E$ represents a unidirectional connection from state x to state y . Let $R_{xy} \in \mathbb{Z}$ denote the reward from state x to state y in an execution sequence. Suppose $R_{xy} = \{0, 1, 50\}$ represents the reward set. If y is a terminal state, then the reward for path e_{xy} is 50; if y is not a terminal state, the reward is 1; otherwise, if path e_{xy} is an anomalous path, the reward is 0.

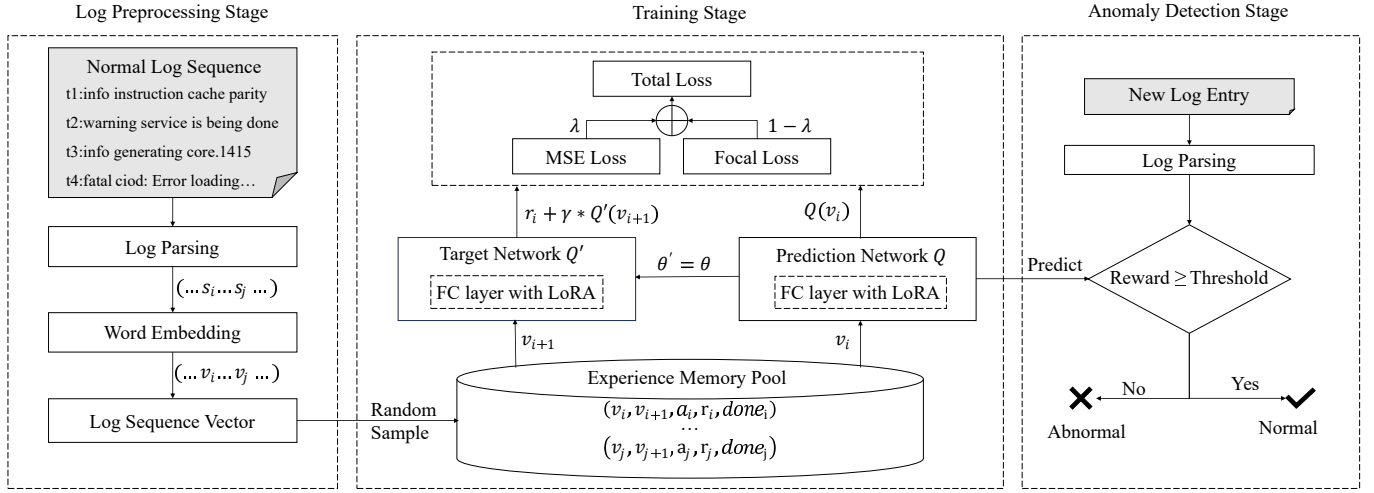


Fig. 1: The framework diagram of DRLLog is mainly divided into three stages: the log parsing and preprocessing stage, the training stage, and the anomaly detection stage. An experience memory pool is constructed from log sequence vectors, from which random samples are extracted for training.

A reinforcement learning model consists of an agent, a set of states, and a set of actions for each state. It employs a policy declared within a Markov Decision Process to execute actions. The agent can transition from one state to another, and executing actions in a given state provides rewards to the agent. Thus, in the context of log anomaly detection, DRLLog can be considered as the agent, where each log data point in the log execution sequence serves as a state S , the action space is represented by the set E , and each execution path e_{xy} in E is an action, with rewards denoted by R_{xy} .

After parsing the raw logs, further preprocessing is performed using Word2Vec to generate log vector representations, thereby capturing additional information from the log sequences. Each log state parsed from the log sequence serves as an input to Word2Vec, mapping each word to a vector space. Assuming the number of distinct log vectors is m , we define $V = \{v_1, v_2, \dots, v_m\}$ as the set of different log vectors.

B. DQN with low-rank adaptation

We propose a log anomaly detection method based on DRL networks. The DRLLog sequence detection network is illustrated in Fig. 2. In DQN, there are two networks with identical structures but different parameters: the prediction network and the target network. The prediction network utilizes the parameters of the latest model to generate the current Q-values, while the target network utilizes the parameters of the previous model to generate the target Q-values. $Q(v, a, r, v')$ represents the output of the prediction network, which evaluates the value function of the current state-action pair. $Q'(v', a', r', v'')$ represents the output of the target network, providing the maximum target Q-value. Therefore, when the agent takes actions in the environment, it can calculate the Q-values according to the formula and update the parameters of the prediction network based on the loss function. After a certain number of iterations, the parameters of the prediction network are copied to the target network. This process is repeated to

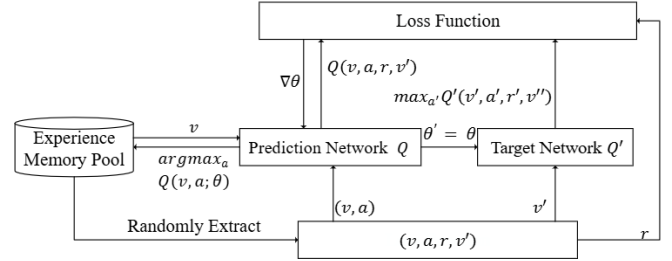


Fig. 2: DRLLog sequence detection network.

continuously train the prediction network. After training, the prediction network is used for anomaly detection on input log sequences.

We are going to build an anomaly detection model based on DRL networks. Firstly, we read the log entries for training to obtain a log state list arranged by ID. We record the execution paths of log states in the log sequence using an experience replay buffer, which includes transitions from the starting state to the destination state, along with the corresponding actions and rewards. Then, we identify all final states in the collection of log sequences and collect the state at each position in the sequence.

We define the agent state as a triple $\text{State} = \langle V, R, E \rangle$, where $V = \{x | \forall x \in V\}$ represents the current log state vector, $R = R_{xy} = \{0, 1, 50\}$ represents the immediate reward received from transitioning from the current log state to the next log state, and $E = \{e_{xy} | e_{xy} \in E, \xi_{xy} = 0 \cup 1, \forall x, y \in V\}$ represents the actions.

During the training stage, the input is the log state at time t , and the output is the log state at time $t+1$. DRLLog adjusts the actions based on feedback rewards. For example, given a log sequence $L = [5, 22, 5, 11, 9, 26]$, if the first log state is 5, DQN outputs a log state s_i . If $s_i = 22$ and matches the next log state in the sequence and is a final state, the reward is 50. If s_i matches the next log state in the sequence but is not a

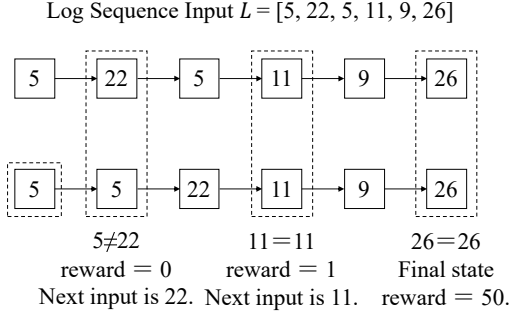


Fig. 3: The training process of the model.

final state, the reward is 1. If s_i does not match the next log state in the sequence, the reward is 0. Then, the actual next log state 22, is used as the next input until the entire log sequence is traversed. The training process is illustrated in Fig. 3.

The process of log state transitions is described as follows: If the current log sequence is $L = [5, 22, 5, 11, 9, 26]$, then the initial state and reward of the agent can be represented as $\text{State} = \langle \{5\}, \{\emptyset\}, \{\emptyset\} \rangle$. The DQN outputs the log state 5, and the agent state is updated to $\text{State} = \langle \{5\}, \langle \{5, 0\} \rangle, \{\emptyset\} \rangle$. Since state 5 and state 22 are different, the reward in the triple is 0. Taking the state 22 as the next input, the output is the log state 5, and the state is immediately updated to $\text{State} = \langle \{22\}, \langle \{5, 1\} \rangle, \langle \{e_{5,22}\} \rangle \rangle$. When the log state is the last state in the log sequence, the agent state becomes $\text{State} = \langle \{26\}, \emptyset, \langle \{e_{5,22}, e_{22,5}, e_{5,11}, e_{11,9}, e_{9,26}\} \rangle \rangle$. A complete execution sequence $H = \langle e_{5,22}, e_{22,5}, e_{5,11}, e_{11,9}, e_{9,26} \rangle$ can be obtained. During the state transition process, the Q-values of both the prediction network and the target network are updated synchronously. However, if the reward of e_{xy} is 0, the Q-value of e_{xy} will not be updated. The update formula is shown in Eq. (1).

$$\text{New}Q(x, e_{xy}) = Q(x, e_{xy}) + \alpha \left[R(x, e_{xy}) + \gamma \max_{e'_{xy}} Q'(x', e'_{xy}) - Q(x, e_{xy}) \right] \quad (1)$$

Where $Q(x, e_{xy})$ denotes the Q-value of the current log state, and α is the learning rate. The learning rate serves as a measure of discarding old values and generating new ones. When the learning rate is 1, it indicates complete abandonment of the old value. $R(x, e_{xy})$ stands for the reward, $\gamma \in [0, 1]$ is a discount factor, and $\max_{e'_{xy}} Q'(x', e'_{xy}) - Q(x, e_{xy})$ represents the path that maximizes the Q-value.

The predicted Q-values are computed using the prediction network, while the target Q-values are computed using the target network. The error between the two is used to update the parameters of the prediction network. After a certain number of iterations, the parameters of the prediction network are copied to the target network, and the training process described above is repeated until all log sequence data have been processed. Once training is complete, log anomaly detection is performed using the prediction network model. The pseudocode of the DRLLog algorithm is described in **Algorithm 1**.

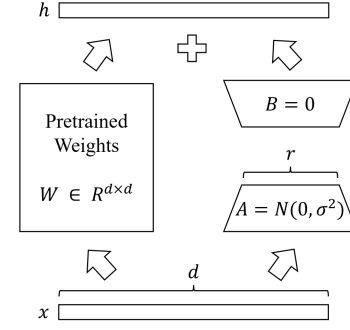


Fig. 4: The Structure of LoRA.

The widespread presence of low-rank structures in machine learning indicates that many problems have inherent low-rank properties. Especially in deep learning tasks, trained neural network models also have low-rank properties. Hu et al. [25] got inspiration from the research of Li et al. [26] and Aghajanyan et al. [27], assuming that the changes in weights during the model adaptation process also have a lower "intrinsic rank", and proposed Low-Rank Adaptation (LoRA). Compared with GPT-3 fine-tuned with Adam [28], LoRA can reduce the number of trainable parameters by 10,000 times and reduce GPU memory requirements by 3 times. And LoRA performs equally well or better than fine-tuning in terms of model quality on RoBERTa [29], DeBERTa [30], GPT-2 and GPT-3.

In the context of DRL networks, the application of LoRA aims to reduce the number of trainable parameters in the model, thereby enhancing the efficiency and generalization performance by adaptively adjusting certain parameters of the weight matrix. The structure of LoRA is shown in Fig. 4.

Since DQN primarily captures contextual semantic features of input sequences through fully connected layers, we apply LoRA to the fully connected layers of both the prediction network and the target network. The prediction network and the target network share the same network structure. Introducing LoRA involves adaptive adjustments to the weight matrix. Specifically, within the fully connected layers of DRL networks, a specific weight matrix W , connecting state representations and action selections, is chosen for adaptation. Taking the weight matrix W as an example, adaptive adjustment is expressed as $W = W_0 + \Delta W$, where $\Delta W = BA$, and B and A represent the low-rank decomposition of the weight matrix. The objective of adaptive adjustment is to optimize the model's performance under the characteristics of downstream tasks through reinforcement learning training, including adjusting specific weight matrices to better adapt to key features of the current task. Through adaptive adjustments to these weights, the model can flexibly adapt to the requirements of different tasks, thereby improving detection performance.

Next, we will provide a detailed description of the network structure for both the original fully connected layer and the fully connected layer integrated with LoRA.

1) *Original Fully Connected Layer*: The fully connected layer is a fundamental component of DRL networks, where each neuron is connected to all neurons in the preceding layer.

Algorithm 1 DRLLog Algorithm Framework

Initialize experience memory pool D with capacity N .
Initialize the target network update frequency C .
Initialize action-value function Q with random weights θ .
Initialize target action-value function $\hat{Q}$ with $\theta' = \theta$.
for episode = 1, ..., M **do**
 Initialize log sequence $v_1 = \{x_1\}$ and preprocess sequence $\phi_1 = \phi(v_1)$.
 for $t = 1, \dots, T$ **do**
 With probability ε , select a random action a_t .
 Otherwise, select $a_t = \arg \max_a Q(\phi(v_1), a; \theta)$.
 Execute action a_t in the environment, obtaining reward r_t and next state x_{t+1} .
 Set $v_{t+1} = v_t, a_t, x_{t+1}$.
 Preprocess $\phi_{t+1} = \phi(v_{t+1})$.
 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D .
 Sample a batch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D with batch size.
 if the episode ends at $j + 1$ **then**
 Set $y_j = r_j$.
 else
 Set $y_j = r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta')$.
 end if
 Perform gradient descent on network parameters θ with respect to $(y_j - Q(\phi_j, a_j; \theta))^2$.
 if t is a multiple of C **then**
 Update $\hat{Q} = Q$.
 end if
 end for
end for

Its output can be computed using Eq. (2).

$$f(x) = \sum_{i=1}^N w_i x_i + b \quad (2)$$

Where N is the number of neurons in the preceding layer, w_i represents the connection weight, x_i denotes the output of the corresponding neuron in the preceding layer, and b is the bias term. Writing this in vector form yields Eq. (3). In this scenario, the fully connected layer requires updating the weight matrix W .

$$f(x) = Wx + b \quad (3)$$

2) *Fully Connected Layer Combined with LoRA*: To reduce training parameters and training costs, we introduce LoRA into the fully connected layer to adaptively perform low-rank decomposition on the connection weight W . We utilize an adaptive low-rank parameter matrix to replace the original connection weights, as shown in Eq. (4).

$$w_i = w_{i0} + \Delta w_i = w_{i0} + b_i a_i \quad (4)$$

Where w_{i0} represents the initial weight, Δw_i denotes the change in the matrix, and b_i and a_i represent the corresponding low-rank parameter matrices when the weight changes. Writing this in vector form yields Eq. (5). In this scenario, only

the low-rank parameter matrices B and A need to be updated, significantly reducing the number of training parameters.

$$f(x) = (W_0 + \Delta W)x + b = (W_0 + BA)x + b \quad (5)$$

These equations describe the mechanism of the original fully connected layer and the fully connected layer integrated with LoRA.

Therefore, introducing LoRA into the structure of DRL networks can effectively reduce the number of parameters in specific weight matrices. By adaptively adjusting weights using the product of low-rank parameter matrices, LoRA helps reduce model complexity and improve training efficiency. Furthermore, LoRA achieves customization for specific tasks through adaptive adjustments, enabling the model to dynamically adapt weights during the learning process to better fit the characteristics of the current task, thereby enhancing performance. Additionally, through adaptive adjustments, the model's generalization ability across different tasks may be improved. LoRA enables the model to flexibly adjust parameters to adapt to changes in various environments and tasks, thereby enhancing the model's versatility.

C. Loss Function

To address the issue of imbalance between normal and abnormal logs during training, we start by introducing the focal loss for binary classification, beginning with the cross-entropy (CE) loss, as shown in Eq. (6).

$$\text{CE}(p, y) = \begin{cases} -\log(p) & \text{if } y = 1 \\ -\log(1 - p) & \text{else} \end{cases} \quad (6)$$

Where $y \in \{0, 1\}$ represents the data category, and $p \in [0, 1]$ is the estimated probability of the model for label $y = 1$. For convenience, we define p_t in Eq. (7).

$$p_t = \begin{cases} p & \text{if } y = 1 \\ 1 - p & \text{else} \end{cases} \quad (7)$$

Substituting p_t into Eq. (6), we obtain Eq. (8) as follows.

$$\text{CE}(p, y) = \text{CE}(p_t) = -\log(p_t) \quad (8)$$

However, even for normal log data, relatively large losses may occur. When summing up the losses for a large number of normal logs, these small losses may dominate, thereby affecting the handling of abnormal log data.

A common method to address data imbalance is by introducing a weighting factor $\alpha \in [0, 1]$, where α is taken for the normal class and $1 - \alpha$ is taken for the abnormal class. In practice, α can be set based on the inverse class frequency or as a hyperparameter adjusted through cross-validation. For better representation, define α_t similar to p_t , and the balanced cross-entropy loss calculation is shown in Eq. (9). Balanced cross-entropy loss is a simple extension of cross-entropy loss and serves as the baseline for focal loss.

$$\text{CE}(p_t) = -\alpha_t \log(p_t) \quad (9)$$

In the actual training process, the problem of class imbalance in logs is not effectively addressed. Although balancing the importance of normal and abnormal logs by introducing the weighting factor α , the difficulty of detecting each log data is not taken into account. Therefore, to more effectively address this issue, by redesigning the loss function, we reduce the weight of normal logs so that the focus of training is more on the more difficult abnormal samples. More specifically, introducing a tunable focusing parameter γ into the cross-entropy loss, where the modulation factor is $(1 - p_t)^\gamma$, with $\gamma \geq 0$. The redefined loss computation formula is shown in Eq. (10).

$$FL(p_t) = -(1 - p_t)^\gamma \log(p_t) \quad (10)$$

Two points are noteworthy: (1) When logs are misclassified and the probability p_t is small, the modulation factor approaches 1, and the loss is not significantly affected. However, as p_t approaches 1, the modulation factor gradually decreases to 0, leading to a reduction in the loss weight of logs that can be correctly identified. (2) The focusing parameter γ smoothly adjusts the reduction rate of the weight of normal logs. When $\gamma = 0$, FL is equivalent to ordinary cross-entropy loss, and as γ increases, the effect of the modulation factor also increases.

Therefore, we combine balanced cross-entropy and focal loss to adopt the balanced focal loss, as shown in Eq. (11). In the experimental process, adopting this form slightly improves model performance compared to the unbalanced form.

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (11)$$

In this paper, in the DRL network, we combine the mean square error (MSE) of the original model with the focal loss proposed in the previous section, balancing the two losses through a weighting coefficient λ .

First, as the DRL updates the model through the outputs of the prediction network Q and the target network Q' , we define MSE to measure the mean square error between the predicted value y_{pred} and the target value y_{target} . Then the MSE loss can be expressed as shown in Eq. (12).

$$\text{MSE}(y_{\text{target}}, y_{\text{pred}}) = \frac{1}{n} \sum_{i=1}^n (y_{\text{target}_i} - y_{\text{pred}_i})^2 \quad (12)$$

Then, we define the focal loss to handle class imbalance issues, making the model focus more on difficult abnormal log data. In the training process, the scarcity of abnormal samples often leads to a model bias towards the features of normal samples. This imbalance makes it challenging to effectively learn the features of abnormal samples, resulting in a higher training loss on them. To mitigate this issue, we can define p_t by Eq. (13).

$$p_t = \frac{1}{1 + |y_{\text{target}} - y_{\text{pred}}|} \quad (13)$$

And focal loss can be defined by Eq. (14).

$$FL(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t) \quad (14)$$

Where α is the balancing factor used to adjust the influence of each class on the loss. γ is the modulation factor of the focal loss, used to adjust the impact of difficult-to-identify logs and easy-to-identify logs on the loss during training. In this way, when there is a large difference between the predicted value y_{pred} and the target value y_{target} due to the scarcity of abnormal samples, p_t is then smaller to obtain a larger loss weight. Additionally, this loss works equally well for normal samples that are harder to train.

Finally, we define the overall loss by a weighted combination of the MSE loss and focal loss. By adjusting the weight coefficient λ to balance the two losses, as shown in Eq. (15).

$$\text{Total Loss} = \lambda \text{MSE}(y_{\text{true}}, y_{\text{pred}}) + (1 - \lambda) FL(p_t) \quad (15)$$

During training, the total loss is used for gradient descent optimization to update the parameters of the DRL network. Considering both mean square error and balanced focal loss ensures that the model can accurately fit the actual values while effectively addressing class imbalance issues, thereby more effectively learning task features and improving model performance.

D. Log Anomaly Detection

This paper focuses on two types of log anomaly detection methods: log point anomalies and log sequence anomalies. Log point anomalies typically represent irregular or deviating entries from the norm. On the other hand, log sequence anomalies indicate that each log in the sequence is normal, but the order of their execution is abnormal. Herein, we will describe how to effectively detect these two types of anomalies.

In a given log execution sequence, DRLLog first utilizes a prediction network model to predict the state of each log. If the prediction result is below the set threshold ϵ , the log state is classified as normal; otherwise, it is classified as an anomalous log state. Subsequently, DRLLog verifies whether the current anomalous log state is included in the set of anomalous states. If it is not in the set, cosine similarity is employed to compute the similarity between the log template of the current state and other normal log templates, comparing the positions of each state.

Let $M = \langle m_1, m_2, \dots, m_i \rangle$ and $N = \langle n_1, n_2, \dots, n_j \rangle$ denote the templates of two log sequences, where M represents the template of the anomalous log sequence and N represents the template of the normal log sequence. The log states in the sequences are generated as log vectors through word embedding, and their similarity is calculated using cosine similarity. Generally, there is a significant difference between anomalous logs and normal logs, and the similarity calculation of log sequence templates is represented by Eq. (16).

$$\text{Cosine Similarity}(M, N) = \frac{A \cdot B}{\|A\| \cdot \|B\|} \quad (16)$$

A lower value of Cosine Similarity(M, N) indicates a greater difference from the log sequence template and a lower probability of anomalies. DRLLog only checks the log

TABLE I: The Datasets.

MDataset	Log Entries	Log States	Anomalies
HDFS	11,117,954	29	16,838
BGL	4,747,963	410	348,460
ThunderBird	20,000,000	1206	764,056

execution order when all log states in the log sequences are normal. For example, consider a given log sequence $L = [5, 22, 5, 11, 9, 26]$. DRLLog first determines whether the log execution path is normal. For instance, when the log state transitions from 5 to 22, the prediction network model provides the Q-value of $e_{5,22}$. If the Q-value is not less than the set threshold ϵ , it indicates that the log execution path is normal; otherwise, it is an anomalous path. In the case where all execution paths are normal, DRLLog checks the status of the last log in the log sequence. If the last state is a final state, the log sequence is considered normal; otherwise, it is an anomalous log sequence. Finally, DRLLog verifies whether the log states in the log sequence are in the correct positions. For example, in usual cases, the position of the final state 26 should be after the fifth position, but in the current log sequence, it is in the sixth position, indicating that the log sequence is abnormal.

IV. EVALUATION

A. Experimental Setup

1) *Dataset*: We use distributed system log datasets as the training and testing sets, including the HDFS dataset, BGL dataset and ThunderBird dataset. The specific information for each dataset is as follows:

- HDFS [31] : This dataset is generated by Amazon running Hadoop-based MapReduce tasks on 203 EC2 nodes and labeled by Hadoop domain experts. It contains 11,197,954 log entries grouped into 575,060 log sequences using block_id, with 16,838 groups of anomalous log sequences.
- BGL [32] : This dataset is generated by LLML's Blue Gene/L supercomputer and contains 1,048,576 log messages, of which 348,460 are anomalous log messages.
- ThunderBird [32] : This dataset is collected from a Thunderbird supercomputer system at Sandia National Labs (SNL) in Albuquerque, with 9,024 processors and 27,072GB memory. It contains more than 200 million log messages, and we leverage its first 20 million consecutive log lines for our experiments, which include 764,056 abnormal log entries.

2) *Metrics*: The paper utilizes four metrics to evaluate the performance of the proposed approach: Precision, Recall, the harmonic mean of Precision and Recall (F1-Score) and ROC-AUC. In the definitions, True Positive (TP) represents correctly detected anomalies, True Negative (TN) represents correctly detected normal logs, False Positive (FP) indicates normal logs incorrectly classified as anomalies, and False Negative (FN) indicates anomalies incorrectly classified as normal logs.

Precision reflects the positive predictive value, i.e., the proportion of correctly detected anomalies among all detected anomalies. Recall reflects the sensitivity, i.e., the proportion of correctly detected anomalies among all actual anomalies. The F1-Score is the harmonic mean of Precision and Recall, used to comprehensively assess the overall performance of the model. These three metrics can be calculated using the following formulas:

$$Precision = \frac{TP}{TP + FP} \quad (17)$$

$$Recall = \frac{TP}{TP + FN} \quad (18)$$

$$F1 - Score = \frac{2 \times (Precision \times Recall)}{(Precision + Recall)} \quad (19)$$

For ROC-AUC, this metric includes two key concepts: the Receiver Operating Characteristic (ROC) curve and the Area Under the Curve (AUC). The horizontal axis of the ROC curve represents the false positive rate (FPR), which is the proportion of actual negative instances that were incorrectly classified as positive. It is calculated as follows:

$$FPR = \frac{FP}{FP + TN} \quad (20)$$

The vertical axis of the ROC curve represents the true positive rate (TPR), which is the proportion of actual positive instances that were correctly classified as positive. It is calculated as follows:

$$TPR = \frac{TP}{TP + FN} \quad (21)$$

The Area Under the Curve (AUC) is defined as the area under the ROC curve. The AUC value reflects the performance of the classifier, with a value closer to 1 indicating a better ability to distinguish between the positive and negative classes.

In practical applications of log anomaly detection, there is often an imbalance between the number of normal and abnormal samples, and the distribution of positive and negative samples can change over time. However, the ROC curve evaluates the classifier's performance at different thresholds and does not depend on the prior probability, which is the actual proportion of positive and negative classes in the dataset. Therefore, the ROC curve remains a meaningful tool for assessing classifier performance, even when the prior probability changes.

3) *Parameters*: The model is built using the PyTorch-based deep learning framework, with Python 3.8.16 and PyTorch 2.0.1 as the runtime environment. The Word2Vec model from Gensim is utilized to generate word embedding. In the initial experimental setup, the input dimension is the number of log states, the action dimension is 1, the hidden layer dimension is also the number of log states, and there are 4 fully connected layers with dimensions of 256, 128, and 64 respectively. The Adam optimizer is employed to optimize the network weights. The training rewards are set to 0, 1, 50, and the threshold parameter ϵ is defaulted to 50. In the focal loss calculation formula, the hyperparameter α is set to 0.75, and the hyperparameter γ is set to 2, which has been empirically

TABLE II: The experimental results on the HDFS dataset.

Method	Precision	Recall	F1-Score	ROC-AUC
PCA [34]	0.981	0.672	0.794	0.681
IM [5]	0.883	0.952	0.911	-
LogCluster [6]	0.870	0.742	0.804	0.552
DeepLog [7]	0.938	0.934	0.936	0.934
LogAnomaly [8]	0.961	0.944	0.955	0.963
QLLog [11]	0.956	0.987	0.968	-
DRLLog	0.997	0.996	0.997	0.989

TABLE III: The experimental results on the BGL dataset.

Method	Precision	Recall	F1-Score	ROC-AUC
PCA [34]	0.501	0.612	0.553	0.821
IM [5]	0.832	0.991	0.912	-
LogCluster [6]	0.425	0.876	0.571	-
DeepLog [7]	0.904	0.965	0.931	0.722
LogAnomaly [8]	0.970	0.942	0.963	0.734
QLLog [11]	0.954	0.996	0.974	-
DRLLog	0.999	1.000	0.999	0.995

validated as an effective combination for classification tasks [33].

4) *Environmental Setting*: All experiments were conducted on a workstation featuring an Intel Core i9-12900H processor, enhanced by an NVIDIA GeForce RTX 3060 Laptop GPU with 16 GB of memory.

5) *Baselines*: On HDFS, BGL and ThunderBird datasets, we compare DRLLog with PCA, IM [5], LogCluster [6], DeepLog [7], LogAnomaly [8] and QLLog [11], using the same data partitioning, experimental parameters, training set, and test set in the experiment. At the same time, for all experiments, we select the best results and record them.

B. Performance Evaluation

This section presents the experimental results of the on-line log anomaly detection model DRLLog based on deep reinforcement learning on the HDFS dataset, BGL dataset and ThunderBird dataset, and compares them with baseline methods longitudinally.

The experimental results of DRLLog on the HDFS dataset are presented in Table II. It can be observed that DRLLog achieves the highest performance on the HDFS dataset, surpassing all baseline methods. The PCA method achieves a Precision on the HDFS dataset that is only 0.016 lower than DRLLog, but its Recall is relatively low at 0.672. This suggests that, compared to DRLLog, the PCA method produces similar false positive predictions on the HDFS dataset, but DRLLog can detect more anomalous logs. IM, Deeplog, and LogAnomaly methods all demonstrate good performance on the HDFS dataset, with F1-Score above 0.9, but lower than QLLog. Deeplog employs log template indexing as input but does not fully consider the semantic information of logs. LogAnomaly focuses more on capturing the semantic information of logs

TABLE IV: The experimental results on the ThunderBird dataset.

Method	Precision	Recall	F1-Score	ROC-AUC
PCA [34]	0.632	0.628	0.629	0.343
IM [5]	0.592	0.903	0.715	-
LogCluster [6]	0.981	0.455	0.621	-
DeepLog [7]	0.964	0.905	0.933	0.957
LogAnomaly [8]	0.890	0.996	0.940	0.961
QLLog [11]	-	-	-	-
DRLLog	0.996	1.000	0.998	0.993

by utilizing single embeddings to learn log representations. QLLog uses reinforcement learning algorithms to learn log sequence patterns, leading to superior performance. Therefore, their performance surpasses that of Deeplog. DRLLog's Precision is 0.041 higher than QLLog, indicating that DRLLog's predictions on normal log sequences are more accurate with fewer false positive predictions, demonstrating the effectiveness of the DRLLog approach on the HDFS dataset.

The experimental results of DRLLog on the BGL dataset are presented in Table III. It can be observed that DRLLog achieves the highest performance on the BGL dataset, surpassing all baseline methods. The IM method exhibits a Recall on the BGL dataset that is only 0.009 lower than DRLLog, but its Precision is relatively lower. This suggests that, compared to DRLLog, the IM method can also detect a considerable number of anomalous logs on the BGL dataset, but IM incorrectly identifies more normal logs as anomalies, resulting in a higher False Positive (FP) value and consequently lower F1-Score. Compared to QLLog and LogAnomaly, DRLLog demonstrates superior performance. This is because LogAnomaly extracts semantic information only from individual log entries, QLLog discretely learns log sequences and records them in a Q-table, while DRLLog learns the patterns of entire log sequences of the dataset, taking the sequence state path as input and directly updating model parameters based on the difference in Q-values between the prediction network and the target network. Therefore, even if the log template index sequence is normal, DRLLog can detect sequence states reflecting anomalies, thereby improving the model's detection performance.

The experimental results of DRLLog on the ThunderBird dataset are presented in Table IV. It can be observed that DRLLog outperforms the baseline methods, achieving the highest Precision, Recall, and F1-Score. This demonstrates that DRLLog can still achieve excellent results on larger-scale datasets.

At the same time, on the three datasets, the ROC-AUC values of DRLLog are closer to 1 compared to all the baseline methods. This indicates that DRLLog can more effectively distinguish abnormal logs from normal logs, handle the issue of class imbalance more efficiently, and perform consistently across different datasets.

Additionally, Table V presents the comparative results between DRLLog and other baseline methods in terms of

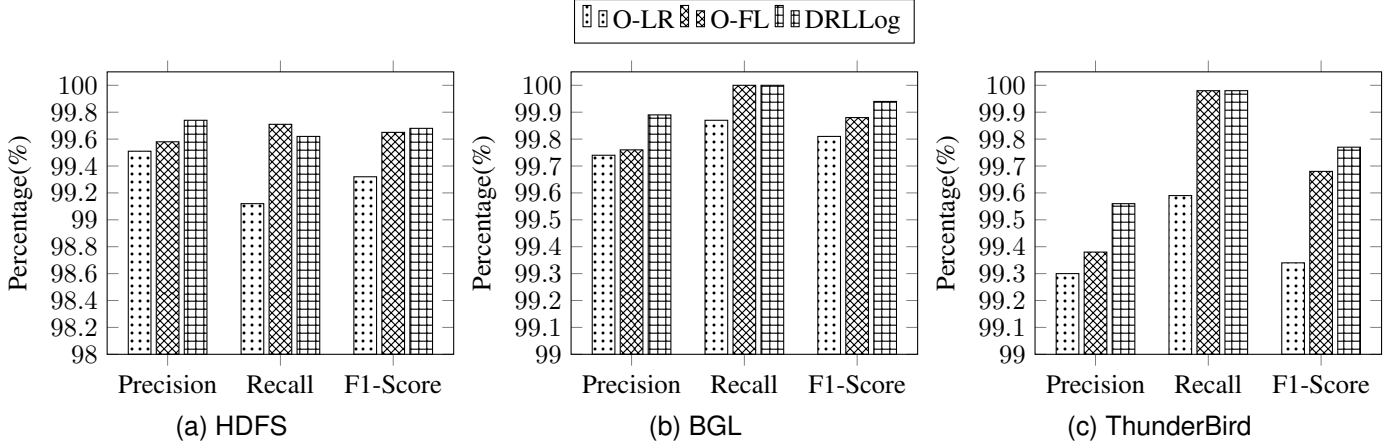


Fig. 5: The ablation experiment results of DRLLog on different datasets. In the histogram, the O-LR indicates that only LoRA is used, the O-FL indicates that only focal loss is used, and the DRLLog indicates the proposed DRLLog method.

TABLE V: The comparative results in terms of parameter, training time per epoch, and testing time.

Method	Parameter	Training time per epoch	Testing time
DeepLog	52,252	15s	283s
LogAnomaly	104,476	32s	516s
QLLog	508,942	51s	1,649s
DRLLog	34,369	7s	157s

parameter count, training time per epoch, and testing time on datasets of the same size. Observing the data, it is evident that DRLLog significantly reduces the parameter count compared to other methods, and also requires significantly less time for both training and testing stages. This is mainly attributed to the introduction of LoRA, which allows training of the decomposed low-rank parameter matrices in the bypass during training and fine-tuning, thereby greatly reducing the number of parameters involved. Moreover, DRLLog exhibits the lowest parameter count compared to other baseline methods, and also demonstrates significantly reduced training and testing times on the same dataset. Therefore, this clearly demonstrates the superiority of LoRA in reducing parameter count and training overhead.

C. Ablation Experiments

This section mainly introduces the comparative analysis of DRLLog method under different compositions and parameters.

In order to explore the contribution of different components to the model performance, ablation experiments were conducted in this section to evaluate the performance of the model under two scenarios: using only low-rank adaptive LoRA and using only focal loss. The results of the ablation experiments on the HDFS, BGL and ThunderBird datasets are shown in Fig. 5 (a), (b) and (c) respectively.

Compared to DRLLog, DRLLog using only LoRA exhibits slightly lower precision and recall on the three datasets; however, the overall decrease in F1-Score is not significant. This

indicates that using LoRA can effectively reduce overhead while maintaining model performance stability, although it may cause minor variations in the model's ability to detect normal and abnormal logs. On the other hand, DRLLog using only focal loss achieves higher Recall, indicating that focal loss can identify more anomalies in the datasets, demonstrating the effectiveness of focal loss in addressing the imbalance issue in log data. Additionally, there is a slight improvement in Precision, which will result in a weak enhancement in the overall performance of the model. In summary, the ablation study confirms the effectiveness of low-rank adaptive and focal loss, which play a crucial role in improving the performance of anomalous log detection.

In this section, experiments were also conducted with different parameter values to explore the influence of each parameter on the experimental results. Fig. 6 (a), (b) and (c) respectively show the evaluation results under different threshold values ϵ on the HDFS dataset, BGL dataset and ThunderBird dataset. It can be observed that on the HDFS dataset, all three metrics remain at a high level when the threshold ϵ is in the range of 57-59; on the BGL dataset, all three metrics also maintain a high level when the threshold ϵ is in the range of 40-44; on the ThunderBird dataset, all three metrics remain at a high level when the threshold ϵ is in the range of 60-68. By adjusting the threshold ϵ , a balance between Precision and Recall can be achieved, resulting in a higher F1-Score.

Fig. 7 (a), (b) and (c) respectively display the evaluation results of DRLLog on the HDFS dataset, BGL dataset and ThunderBird dataset using different proportions of training data to simulate online anomaly detection. It can be observed that on the HDFS dataset, all three metrics remain at a high level when the proportion of training data exceeds 10%, and even at lower proportions of training data, the metrics can still be maintained at over 97%. On the BGL dataset, when the proportion of training data exceeds 10%, all three metrics also reach a high level. Although at lower proportions of training data, the Recall is relatively lower, and cannot effectively identify abnormal log sequences, the Precision

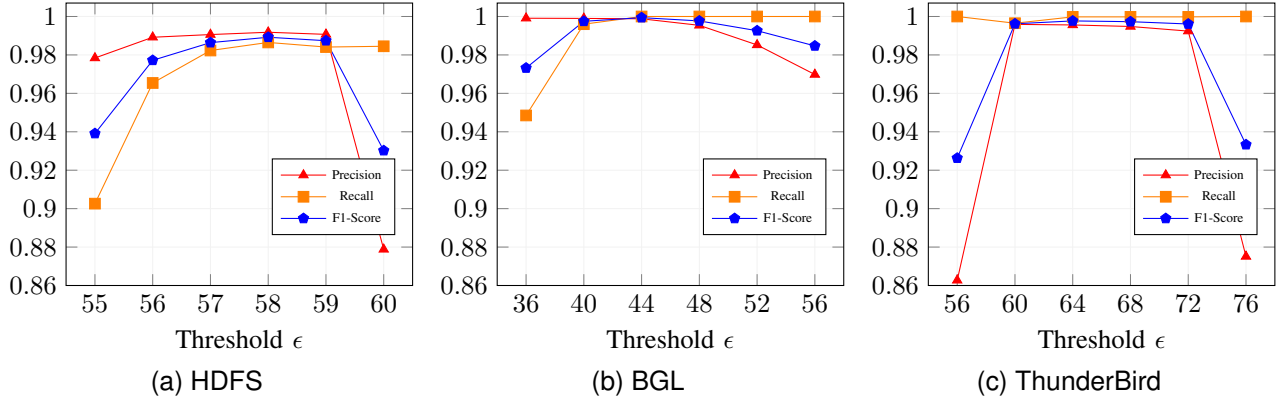


Fig. 6: The experimental result graphs under different thresholds ϵ .

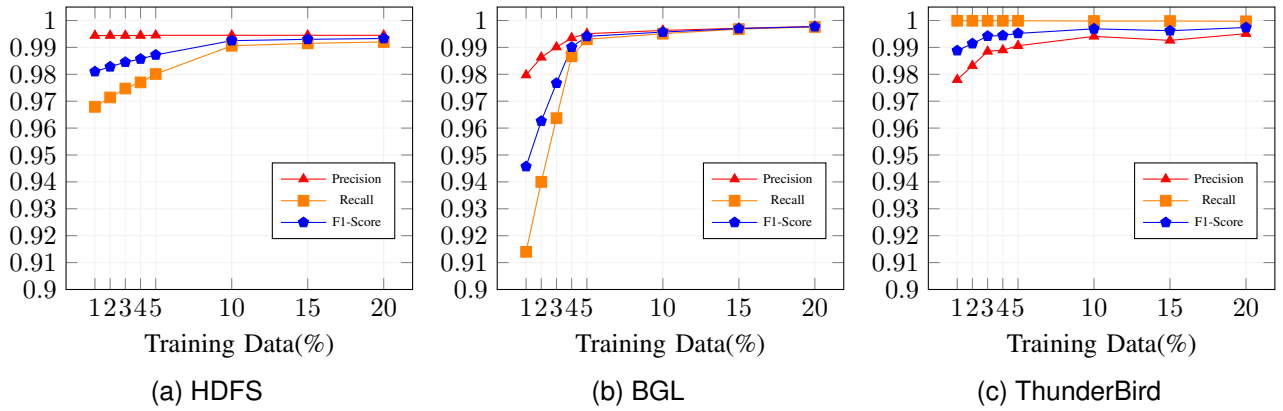


Fig. 7: The experimental result graphs under different proportions of training data.

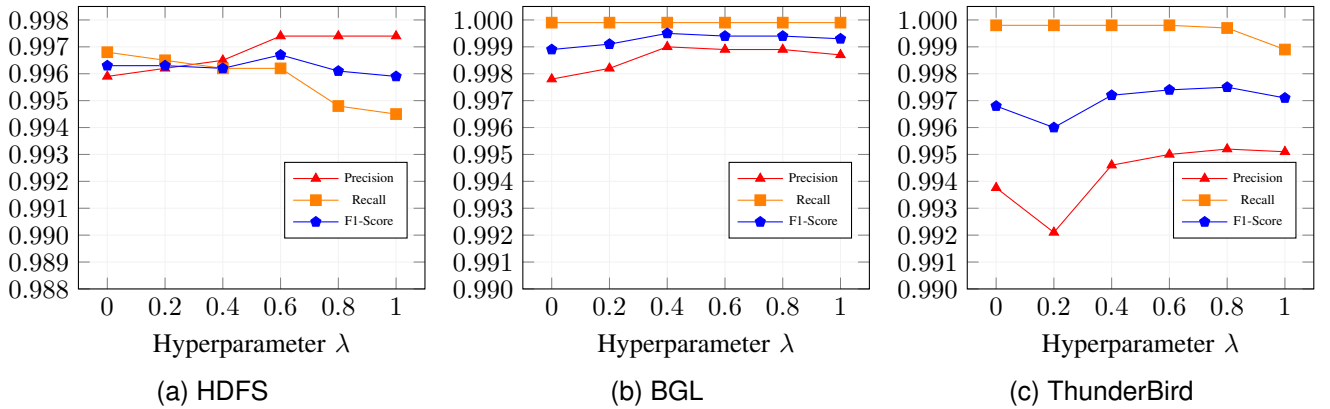


Fig. 8: The experimental result graphs under different hyperparameter λ of loss function.

remains consistently high. As for the ThunderBird dataset, these three metrics maintain a high level when the proportion of training data exceeds 10%, and the metrics can remain above 98% even when the proportion of training data is below 5%. In conclusion, with an increase in log data, DRLLog can maintain a high level across all three metrics, and the detection accuracy does not decrease. It can effectively adapt to new log states and patterns, thereby achieving efficient online anomaly

detection.

Fig. 8 (a), (b), and (c) respectively display the evaluation results of DRLLog on the HDFS dataset, BGL dataset, and ThunderBird dataset using different values of the hyperparameter λ to explore the influence of mean square error (MSE) and focal loss on the experimental results. When λ equals 1, it indicates that only the MSE loss is used. Conversely, when λ equals 0, it indicates that only the focal loss is used.

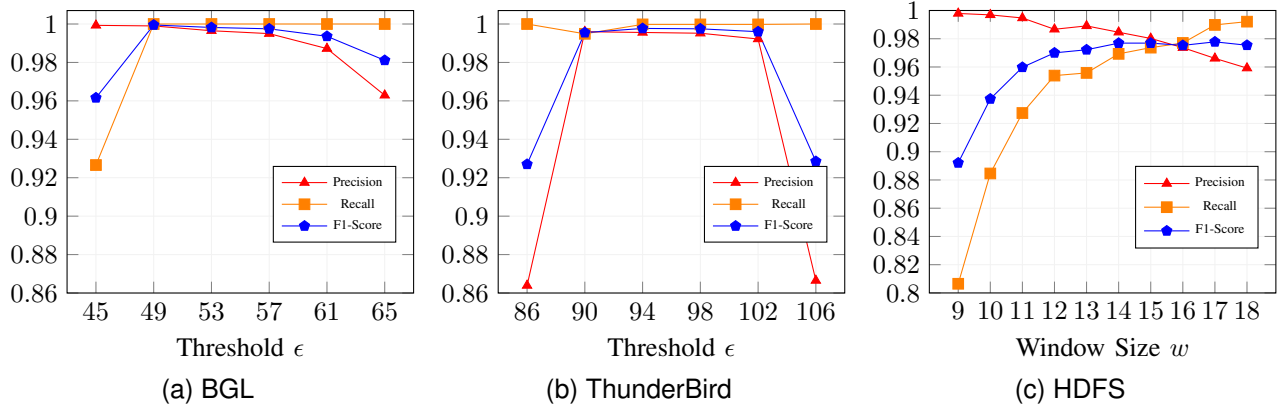


Fig. 9: (a) and (b) show the experimental result graphs under linear reward function on BGL dataset and ThunderBird dataset respectively, while (c) shows the experimental result graph under different window sizes on the HDFS dataset.

It can be observed that the use of focal loss ensures that the model maintains a high Recall across the three datasets. By combining focal loss with mean square error loss, the model achieves a better balance between Precision and Recall, resulting in a higher F1-Score.

D. Discussion On Different Reward Functions

This section mainly discusses the impact of various designed reward functions on the performance of the DRLLog method. Besides setting the reward to 0, 1, 50 in the training stage, we also design a dynamic linear reward function as:

$$R(t) = \frac{t}{T} + b \quad (22)$$

Here, t represents the position of the current log state within the sequence, T denotes the total length of the log sequence, and b is an offset set to 1 by default, ensuring that even the first step receives some base reward. The purpose of this reward function is to explore whether the position of the log within the sequence affects the performance of the model. Specifically, during the training phase, execution paths that are closer to the final state of the log sequence receive higher rewards. And also 50 is set as a final reward for the execution path of the normal log to reach a final state, while 0 is set to indicate an anomalous path. The experimental results are presented in Fig. 9.

It can be observed that the introduction of a dynamic linear reward function on the BGL dataset and the ThunderBird dataset does not significantly improve the experimental results. However, because the reward obtained from the execution path between each log state increases, the threshold interval over which the model performs well increases accordingly. From the analysis of the experimental results, it can be concluded that the position information of the log state in the sequence does not significantly improve the performance of DRLLog.

E. Discussion on Different Formats of Log State

In this section, a different format of log state is discussed to assess the model's robustness and adaptability. We choose the

HDFS dataset because HDFS collects abnormal and normal log data by block_id, rather than by individual log entries, as is the case with the BGL and ThunderBird datasets. Consequently, the HDFS dataset is more likely to contain log sequences where each log entry is normal, but their execution order is abnormal. To capture such abnormal instances, we use a sliding window approach to define log states, which allows us to obtain richer sequential pattern information. In other words, let $L = [l_1, l_2, \dots, l_n]$ be a series of log entries, and let the size of the sliding window be w . Then for a window starting at position t , the set of log entries it contains can be defined as $L_t = [l_t, l_{t+1}, \dots, l_{t+w-1}]$. In this context, the log data within each window in the log sequence is served as a state s . In contrast, the original method serves each log data point in the log sequence as a state s . Let $V_t = [v_t, v_{t+1}, \dots, v_{t+w-1}]$ be the log vectors within the window at position t , where v_i is the log vector representation obtained using Word2Vec, then we can get the representation of state s at position t as follows:

$$s_t = f_{LSTM}(V_t) \quad (23)$$

And the next state of s_t can be represented as:

$$s_{t+1} = f_{LSTM}(V_{t+1}) \quad (24)$$

Fig. 9 (c) displays the experimental results for different window sizes on the HDFS dataset. It can be observed that as the window size w increases, a balance between Precision and Recall is achieved, leading to a higher F1-Score in the range of 14-17, which is around 0.977. Although the use of a sliding window causes the final F1-Score to drop by approximately 0.02, it remains 0.012 higher than the baseline model QLLog. The experimental results demonstrate that DRLLog achieves a notable level of robustness and adaptability in sequential pattern based anomaly detection.

F. Discussion on Real-world Application of DRLLog

Deploying DRLLog in production poses several challenges. For example, in practical distributed and parallel systems, the dynamic nature of log sequences, the presence of noise in

log data, and the concurrency of multiple exceptions introduce more uncertainty into the state transition patterns derived from log sequence analysis [35]. This increased uncertainty can potentially affect the practical effectiveness of DRLLog. One possible solution is to convert the log sequence into a directed graph structure, thereby reducing the model's dependence on specific log sequence patterns.

V. CONCLUSION

This paper introduces a novel log anomaly detection method called DRLLog, which utilizes deep reinforcement learning networks to effectively identify anomalous behaviors in system logs. DRLLog leverages Deep Q Network to make decisions and learn in complex and high-dimensional log spaces, while effectively reducing model overhead through low-rank adaptation techniques. By introducing focal loss, DRLLog can concentrate more on learning the features of abnormal logs, thereby addressing the issue of imbalanced quantities between normal and abnormal logs. Experimental results demonstrate that DRLLog outperforms mainstream log anomaly detection methods on multiple log datasets, with reduced parameter count and training time, showcasing high efficiency and feasibility during model training.

VI. ACKNOWLEDGMENT

This work was partially supported by the National Key Research and Development Program (Grant No. 2022YFB3104001), the Key Research and Development Program of Hubei Province (Grant No. 2022BAA050).

REFERENCES

- [1] X. Zhao, Z. Jiang, and J. Ma, "A survey of deep anomaly detection for system logs," *2022 International Joint Conference on Neural Networks*, pp. 1–8, 2022.
- [2] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, p. 100470, 2023.
- [3] P. Ryciak, K. Wasielewska, and A. Janicki, "Anomaly detection in log files using selected natural language processing methods," *Applied Sciences*, p. 5089, 2022.
- [4] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "Swisslog: Robust anomaly detection and localization for interleaved unstructured logs," *IEEE Transactions on Dependable and Secure Computing*, pp. 2762–2780, 2023.
- [5] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, "Mining invariants from console logs for system problem detection," *2010 USENIX Annual Technical Conference*, p. 24, 2010.
- [6] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," *Proceedings of the 38th International Conference on Software Engineering Companion*, pp. 102–111, 2016.
- [7] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, pp. 1285–1298, 2017.
- [8] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun *et al.*, "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pp. 4739–4745, 2019.
- [9] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li, J. Chen, X. He, R. Yao, J.-G. Lou, M. Chintalapati, F. Shen, and D. Zhang, "Robust log-based anomaly detection on unstable log data," *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 807–817, 2019.
- [10] K. Arshad, R. F. Ali, A. Muneer, I. A. Aziz, S. Naseer, N. S. Khan, and S. M. Taib, "Deep reinforcement learning for anomaly detection: A systematic review," *IEEE Access*, pp. 124 017–124 035, 2022.
- [11] X. Duan, S. Ying, W. Yuan, H. Cheng, and X. Yin, "Qlog: A log anomaly detection method based on q-learning algorithm," *Information Processing & Management*, p. 102540, 2021.
- [12] A. T. Tunkiel, D. Sui, and T. Wiktorski, "Impact of data pre-processing techniques on recurrent neural network performance in context of real-time drilling logs in an automated prediction framework," *Journal of Petroleum Science and Engineering*, p. 109760, 2022.
- [13] B. Wang, A. Wang, F. Chen, Y. Wang, and C.-C. J. Kuo, "Evaluating word embedding models: Methods and experimental results," *APSIPA transactions on signal and information processing*, p. e19, 2019.
- [14] G. Di Gennaro, A. Buonanno, and F. A. Palmieri, "Considerations about learning word2vec," *The Journal of Supercomputing*, pp. 1–16, 2021.
- [15] R. Brochier, A. Guille, and J. Velcin, "Global vectors for node representations," *The World Wide Web Conference*, pp. 2587–2593, 2019.
- [16] A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, and T. Mikolov, "Fasttext. zip: Compressing text classification models," *arXiv preprint arXiv:1612.03651*, 2016.
- [17] H.-n. Wang, N. Liu, Y.-y. Zhang, D.-w. Feng, F. Huang, D.-s. Li, and Y.-m. Zhang, "Deep reinforcement learning: A survey," *Frontiers of Information Technology & Electronic Engineering*, pp. 1726–1744, 2020.
- [18] P. Ladosz, L. Weng, M. Kim, and H. Oh, "Exploration in deep reinforcement learning: A survey," *Information Fusion*, pp. 1–22, 2022.
- [19] Y.-F. Hsu and M. Matsuoka, "A deep reinforcement learning approach for anomaly network intrusion detection system," *2020 IEEE 9th international conference on cloud networking*, pp. 1–6, 2020.
- [20] G. Pang, A. van den Hengel, C. Shen, and L. Cao, "Toward deep supervised anomaly detection: Reinforcement learning from partially labeled anomaly data," *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pp. 1298–1308, 2021.
- [21] T. Wu and J. Ortiz, "Rlad: Time series anomaly detection through reinforcement learning and active learning," *arXiv preprint arXiv:2104.00543*, 2021.
- [22] D. Zha, K.-H. Lai, M. Wan, and X. Hu, "Meta-aad: Active anomaly detection with deep reinforcement learning," *2020 IEEE International Conference on Data Mining*, pp. 771–780, 2020.
- [23] M. Botvinick, J. X. Wang, W. Dabney, K. J. Miller, and Z. Kurth-Nelson, "Deep reinforcement learning and its neuroscientific implications," *Neuron*, pp. 603–616, 2020.
- [24] S. E. Li, "Deep reinforcement learning," *Reinforcement Learning for Sequential Decision and Optimal Control*, pp. 365–402, 2023.
- [25] E. J. Hu, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models," *International Conference on Learning Representations*, 2021.
- [26] C. Li, H. Farkhoor, R. Liu, and J. Yosinski, "Measuring the intrinsic dimension of objective landscapes," *International Conference on Learning Representations*, 2018.
- [27] A. Aghajanyan, S. Gupta, and L. Zettlemoyer, "Intrinsic dimensionality explains the effectiveness of language model fine-tuning," *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 7319–7328, 2020.
- [28] D. Kingma, "Adam: A method for stochastic optimization," *International Conference on Learning Representations*, 2014.
- [29] Z. Liu, W. Lin, Y. Shi, and J. Zhao, "A robustly optimized bert pre-training approach with post-training," *China National Conference on Chinese Computational Linguistics*, pp. 471–484, 2021.
- [30] P. He, X. Liu, J. Gao, and W. Chen, "Deberta: Decoding-enhanced bert with disentangled attention," *International Conference on Learning Representations*, 2020.
- [31] M. R. Ghazi and D. Gangodkar, "Hadoop, mapreduce and hdfs: A developers perspective," *Procedia Computer Science*, pp. 45–50, 2015.
- [32] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," *37th annual IEEE/IFIP international conference on dependable systems and networks*, pp. 575–584, 2007.
- [33] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *2017 IEEE International Conference on Computer Vision*, pp. 2999–3007, 2017.
- [34] C. Labrin and F. Urdinez, "Principal component analysis," *R for Political Data Science*, pp. 375–393, 2020.
- [35] J. Li, H. He, S. Chen, D. Jin, and J. Yang, "Loggraph: Log event graph learning aided robust fine-grained anomaly diagnosis," *2024 IEEE Transactions on Dependable and Secure Computing*, pp. 1876–1889, 2024.



Junwei Zhou (Member, IEEE) received the Ph.D. degree from the Department of Electrical Engineering, City University of Hong Kong. Since then, he has been a Postdoctoral Researcher with Pennsylvania State University, USA, and a visiting scholar at the Polytechnic University of Turin, Italy. He is currently a Professor with the School of Computer and Artificial Intelligence, Wuhan University of Technology, China. His research interests include computer vision, information security, and distributed source coding. He has served as a PC Member for top

conferences, such as EMNLP 2023, Coling 2022, ACL 2022, and ISSRE 2022.



Cheng Tan is the lecturer with the School of Computer Science and Artificial Intelligence at Wuhan University of Technology, China. He received both the B.Sc. and Ph.D. degrees in information security from Wuhan University, China, in 2011, and 2018, respectively. His research interests include system security, network security, smart contract security, and blockchain security.



Yuyang Gao received the B.S. degree in computer science and technology from the Wuhan University of Technology, where he is currently pursuing the M.S. degree in computer technology. His current research interests include log anomaly detection and vulnerability detection.



Ying Zhu received the B.S. degree in computer science and technology from the Zhongnan University of Economics and Law, and she is currently pursuing the M.S. degree in computer technology at Wuhan University of Technology. Her current research interests include log anomaly detection and vulnerability detection.



Jianwen Xiang (Member, IEEE) received the Ph.D. degree in computer software and theory from Wuhan University and in information science from the Japan Advanced Institute of Science and Technology. Since then, he has been a Special Research Fellow with the National Institute of Advanced Industrial Science and Technology, Japan, the Director of the Central Research Laboratory, NEC Corporation, Japan, and a Postdoctoral Researcher with the Japan Advanced Institute of Science and Technology. He is currently a Professor with the School of Computer Science

and Technology, Wuhan University of Technology, China, and a Visiting Professor with Kyushu University, Japan. His research interests include dependable computing, information security, formal methods, and knowledge management. He has served as a Reviewer for international journals, including IEEE Transactions on Reliability, Reliability Engineering and System Safety, ACM Transactions on Software Engineering and Methodology, and Journal of Systems and Software.



Xiangtian Yu received the B.S. degree in computer science and technology from the Wuhan University of Technology, where he is currently pursuing the M.S. degree in computer technology. His current research interests include log anomaly detection, system reliability, and electronic information.



Yanchao Yang received the MA.SC. from the department of Artificial Intelligence, Beijing Normal University. She is currently pursuing a Ph.D. degree at Wuhan University of Technology. Her research interests include computer vision, deep learning, information security.